



Module: Gestion des permissions

Systeme Linux



Stella Roulière

Objectifs



À l'issue de ce cours, les apprenants seront capables de :

- ✓ Comprendre et manipuler les droits d'accès Linux (lecture, écriture, exécution) pour les propriétaires, les groupes et les autres utilisateurs.
- ✓ Utiliser la commande `ls -l` pour afficher les permissions des fichiers et répertoires.
- ✓ Modifier les permissions des fichiers et répertoires à l'aide de la commande `chmod` avec les notations symbolique et numérique.
- ✓ Modifier le propriétaire et le groupe d'un fichier ou répertoire avec la commande `chown`.
- ✓ Configurer le `umask` pour définir les permissions par défaut lors de la création de fichiers et de répertoires.
- ✓ Appliquer et comprendre l'impact des droits spéciaux : SetUID, SetGID et Sticky Bit.
- ✓ Gérer les Access Control Lists (ACL) avec les commandes `getfacl` et `setfacl` pour accorder des permissions à des utilisateurs et groupes spécifiques.

Sommaire



Gestion des permissions

Page 4

UMASK

Page 12

Droits spéciaux

Page 16

ACL

Page 22

Gestion des permissions





Gestion des permissions

Les droits d'accès Linux, également appelés permissions, déterminent qui peut accéder, modifier ou exécuter des fichiers et des répertoires sur un système Linux.

Il existe trois types de permissions (droits) pour chaque fichier ou répertoire : lecture, écriture et exécution.

Ces permissions sont attribuées à trois catégories d'utilisateurs : le propriétaire du fichier, le groupe associé au fichier et les autres utilisateurs.

	Fichier	Répertoire
R (Lecture)	Permet la lecture d'un fichier	Permet de lister des fichiers dans un répertoire.
W (Écriture)	Pour les fichiers, permet la modification du contenu	Permet la modification ou la création de nouveaux fichiers dans un répertoire
X (Exécution)	Permet l'exécution	L'accès à un répertoire



Gestion des permissions

Pour afficher les droits d'un fichier ou répertoire la commande `ls -l` sera utilisée.

```
(stella@K-Rourou)-[~/Documents]
$ ls -l
total 8
-rw-r--r-- 1 stella stella 3734 Jan 17 09:32 back-passwd.txt
drwxr-xr-x 2 stella stella 4096 Jan 17 09:33 Procédures

(stella@K-Rourou)-[~/Documents]
$
```

U - l'utilisateur propriétaire stella a les droits d'afficher et modifier le fichier

u+g+o=a

-	r	w	-	r	-	-	r	-	-	1	stella	stella	3734	Jan	17	09:32	back-passwd.txt
											u	g	o				

O - Les autres utilisateurs du système ont le droit d'afficher le fichier

g - le groupe propriétaire stella à les droits d'afficher fichier



Gestion des permissions

La commande : chmod

La commande est employée pour modifier les permissions (droits d'accès) d'un fichier ou d'un répertoire.

Elle permet de définir qui peut lire, écrire ou exécuter un fichier, et elle est souvent utilisée avec des codes numériques (en base 8) ou des notations symboliques pour configurer ces permissions (rwx).

Syntaxe: `chmod [options] permissions fichier`

L'option -R : Applique le changement récursivement à tous les fichiers et sous-répertoires d'un répertoire.

r	w	x	Total	
4	2	1	7	(rwx)
4	2	0	6	(rw-)
4	0	1	5	(r-x)
4	0	0	4	(r--)
0	2	1	3	(-wx)
0	2	0	2	(-w-)
0	0	1	1	(--x)
0	0	0	0	(---)



Gestion des permissions

Modifier les permissions d'un répertoire en notation symbolique :

```
(stella@K-Rourou)-[~/Documents]
$ ls -l
total 8
-rw-r--r-- 1 stella stella 3734 Jan 17 09:32 back-passwd.txt
drwxr-xr-x 2 stella stella 4096 Jan 17 09:33 Procédures

(stella@K-Rourou)-[~/Documents]
$ chmod -R g+w,o-rx Procédures
```

```
(stella@K-Rourou)-[~/Documents]
$ ls -l
total 8
-rw-r--r-- 1 stella stella 3734 Jan 17 09:32 back-passwd.txt
drwxrwx--- 2 stella stella 4096 Jan 17 09:33 Procédures
```




Gestion des permissions

On va remettre les permissions du répertoire en notation numérique :

```
(stella@K-Rourou)-[~/Documents]
$ ls -l
total 8
-rw-r--r-- 1 stella stella 3734 Jan 17 09:32 back-passwd.txt
drwxrwx— 2 stella stella 4096 Jan 17 09:33 Procédures
```

```
(stella@K-Rourou)-[~/Documents]
$ chmod -R 755 Procédures
```

```
(stella@K-Rourou)-[~/Documents]
$ ls -l
total 8
-rw-r--r-- 1 stella stella 3734 Jan 17 09:32 back-passwd.txt
drwxr-xr-x 2 stella stella 4096 Jan 17 09:33 Procédures
```

```
(stella@K-Rourou)-[~/Documents]
$
```



Gestion des permissions

On peut modifier les permissions en notation symbolique aussi comme cela:

```
(stella@K-Rourou)-[~/Documents]
$ ls -l
total 8
-rw-r--r-- 1 stella stella 3734 Jan 17 09:32 back-passwd.txt
drwxr-xr-x 2 stella stella 4096 Jan 17 09:33 Procédures

(stella@K-Rourou)-[~/Documents]
$ chmod u=rwx,g=rw back-passwd.txt

(stella@K-Rourou)-[~/Documents]
$ ls -l
total 8
-rwxrw-r-- 1 stella stella 3734 Jan 17 09:32 back-passwd.txt
drwxr-xr-x 2 stella stella 4096 Jan 17 09:33 Procédures

(stella@K-Rourou)-[~/Documents]
$
```



Gestion des permissions

La commande : chown

Elle est utilisée pour changer le propriétaire et/ou le groupe d'un fichier ou d'un répertoire.

Le changement de propriétaire peut être utile dans divers scénarios, notamment lors de la migration de fichiers entre utilisateurs, de la restauration des propriétaires après une restauration à partir d'une sauvegarde, etc.

Syntaxe: **chown [options] user:group fichier/dossier**

-R : Applique le changement récursivement à tous les fichiers et sous-répertoires d'un répertoire

```
(stella@K-Rourou)-[~/Documents]
$ ls -l
total 8
-rwxrw-r-- 1 stella stella 3734 Jan 17 09:32 back-passwd.txt
drwxr-xr-x 2 stella stella 4096 Jan 17 09:33 Procédures

(stella@K-Rourou)-[~/Documents]
$ sudo chown user1:user2 back-passwd.txt
[sudo] password for stella:

(stella@K-Rourou)-[~/Documents]
$ ls -l
total 8
-rwxrw-r-- 1 user1 user2 3734 Jan 17 09:32 back-passwd.txt
drwxr-xr-x 2 stella stella 4096 Jan 17 09:33 Procédures
```

Ci-dessous je modifie uniquement le groupe avec l'option : devant le nom du groupe

```
(stella@K-Rourou)-[~/Documents]
$ sudo chown :user4 Procédures

(stella@K-Rourou)-[~/Documents]
$ ls -l
total 8
-rwxrw-r-- 1 user1 user2 3734 Jan 17 09:32 back-passwd.txt
drwxr-xr-x 2 stella user4 4096 Jan 17 09:33 Procédures
```


UMASK





UMASK

La commande : umask

Le umask (masque de permissions) est une commande et un concept des systèmes d'exploitation Linux. Il sert à définir les permissions par défaut des fichiers et répertoires créés par un utilisateur. Plus précisément, le umask indique quels bits de permission doivent être retirés des permissions par défaut lors de la création.

Par exemple, avec un umask par défaut de 022, les nouveaux fichiers auront des permissions 644 (rw-r--r--) et les répertoires 755 (rwxr-xr-x). Pour personnaliser le umask à chaque connexion, il est possible d'ajouter une commande umask dans le fichier de configuration du shell utilisé, comme ~/.bashrc pour le shell Bash.

```
(stella@K-Rourou) - [~/Documents]  
$ umask  
022
```



UMASK

La commande : umask par défaut

```
Trash
(stella@K-Rourou) - [~/Documents]
$ umask
022

File System
(stella@K-Rourou) - [~/Documents]
$ mkdir doc1 ; touch fic1

Home
(stella@K-Rourou) - [~/Documents]
$ ls -l
total 12
-rwxrw-r-- 1 user1 user2 3734 Jan 17 09:32 back-passwd.txt
drwxr-xr-x 2 stella stella 4096 Jan 17 10:53 doc1
-rw-r--r-- 1 stella stella 0 Jan 17 10:53 fic1
drwxr-xr-x 2 stella user4 4096 Jan 17 09:33 Procédures
```

	Pour un dossier			
Droits max	7	7	7	
umask	0	2	2	
On va soustraire				
	7	5	5	
	(rwx)	(r-x)	(r-x)	

	Pour un fichier			
Droits max	6	6	6	
umask	0	2	2	
On va soustraire				
	6	4	4	
	(rw-)	(r--)	(r--)	



UMASK

La commande : umask modification

	Pour un dossier			
Droits max	7	7	7	
umask	0	4	4	
	On va soustraire			
	7	3	3	
	(rw-)	(-wx)	(-wx)	

	Pour un fichier			
Droits max	6	6	6	
umask	0	6	6	
	On va soustraire			
	6	0	0	
	(rw-)	(---)	(---)	

```
(stella@K-Rourou)-[~/Documents/Procédures]
$ umask 044

(stella@K-Rourou)-[~/Documents/Procédures]
$ mkdir doc3

(stella@K-Rourou)-[~/Documents/Procédures]
$ ls -l
total 4
drwx-wx-wx 2 stella stella 4096 Jan 17 11:01 doc3
```

```
(stella@K-Rourou)-[~/Documents/Procédures]
$ umask 066

(stella@K-Rourou)-[~/Documents/Procédures]
$ touch fic3

(stella@K-Rourou)-[~/Documents/Procédures]
$ ls -l fic3
-rw----- 1 stella stella 0 Jan 17 10:58 fic3
```

Droits spéciaux





Droits spéciaux

En complément des permissions classiques (lecture, écriture, exécution) attribuées au propriétaire, au groupe et aux autres utilisateurs, les systèmes de fichiers Unix/Linux permettent d'appliquer trois attributs spécifiques aux fichiers et répertoires : SetUID (Set User ID), SetGID (Set Group ID) et Sticky Bit.

	Fichier	Répertoire
SetUID (SUID) : chmod u+s programme chmod 4755 programme	Lorsqu'un fichier a l'attribut SetUID activé, il s'exécute avec les permissions du propriétaire du fichier, plutôt qu'avec celles de l'utilisateur qui lance l'exécution. Par exemple, la commande passwd utilise ce mécanisme.	Il est généralement ignoré.
SetGID (SGID) : chmod g+s repertoire ou fichier chmod 2775	Quand l'attribut SetGID est activé, le processus s'exécute avec les droits du groupe propriétaire du fichier, et non ceux du groupe de l'utilisateur qui l'exécute. Cela permet de donner à un groupe des permissions spécifiques sur un fichier. Par exemple le fichier /etc/hosts	Si SetGID est activé sur un répertoire, tout nouveau fichier créé dans ce répertoire héritera du groupe de ce répertoire, et non du groupe de l'utilisateur qui l'a créé. Cela assure une homogénéité des appartenances de groupe au sein du répertoire.
Sticky Bit : chmod +t repertoire chmod 1777 repertoire	Sticky Bit activé, pas d'effet significatif	Avec le Sticky Bit activé sur un répertoire, seul le propriétaire d'un fichier peut le supprimer ou le renommer dans ce répertoire, même si d'autres utilisateurs disposent des droits d'écriture sur ce répertoire.



Droits spéciaux

SetUID

```
Trash  
[stella@K-Rourou] - [~/Documents/Procédures]  
$ sudo ls -l /usr/bin/passwd  
[sudo] password for stella:  
-rwsr-xr-x 1 root root 72344 Oct 15 13:10 /usr/bin/passwd  
[stella@K-Rourou] - [~/Documents/Procédures]  
$
```

Ici le propriétaire est root et cette commande vous l'exécutez en tant que root quand vous changez votre mot de passe. Le s minuscule indique que vous avez les droits exécutions (x). Le S majuscule indiquerait qu'il n'y a pas les x.



Droits spéciaux

GUID

```
stella@DB-CLT:~$ mkdir commun
```

```
drwxr-xr-x 2 stella stella 4096 3 avril 10:59 commun
```

```
stella@DB-CLT:~$ chmod g+s commun/
```

```
chown :toto /home/stella/commun/
```

```
drwxr-sr-x 2 stella toto 4096 3 avril 10:59 commun
```

Maintenant tous les fichiers
créés dans commun hériteront
du groupe toto

```
stella@DB-CLT:~/commun$ touch toto
stella@DB-CLT:~/commun$ ls -l
total 0
-rw-r--r-- 1 stella toto 0 3 avril 11:04 toto
stella@DB-CLT:~/commun$
```



Droits spéciaux

Sans Sticky bit

```
root@DB-CLT:/# mkdir public
root@DB-CLT:/# chmod 757 public/
root@DB-CLT:/#
```

```
drwxr-xrwx  2 root root  4096  3 avril 11:19 public
```

```
stella@DB-CLT:/$ cd public/
stella@DB-CLT:/public$ touch test2
stella@DB-CLT:/public$ ls -l test2
-rw-r--r-- 1 stella stella 0  3 avril 11:20 test2
stella@DB-CLT:/public$ su toto
Mot de passe :
toto@DB-CLT:/public$ ls
test2
toto@DB-CLT:/public$ rm test2
rm : supprimer 'test2' qui est protégé en écriture et est du type « fichier vid
» ? oui
toto@DB-CLT:/public$ ls
toto@DB-CLT:/public$
```

Ici Stella crée un fichier dans le dossier Public, elle est propriétaire

Toto peut supprimer le fichier de Stella



Droits spéciaux

Avec Sticky bit

```
root@DB-CLT:/# pwd  
/  
root@DB-CLT:/# mkdir commun
```

```
drwxr-xr-x  2 root root 4096  3 avril 11:08 commun
```

```
root@DB-CLT:/# chmod o+wt commun/ ou root@DB-CLT:/# chmod 1757 commun/
```

```
drwxr-xrwt  2 root root 4096  3 avril 11:08 commun
```

```
stella@DB-CLT:/commun$ touch test  
stella@DB-CLT:/commun$ ls -l  
total 0  
-rw-r--r--  1 stella stella 0  3 avril 11:14 test
```

```
toto@DB-CLT:/commun$ rm test  
rm : supprimer 'test' qui est protégé en écriture et est du  
» ? oui  
rm: impossible de supprimer 'test': Opération non permise  
toto@DB-CLT:/commun$
```

**Toto ne peut plus
supprimer le fichier de
Stella**

ACL





ACL

La commande `getfacl` est utilisée sous Linux pour afficher les contrôles d'accès (ACL, Access Control Lists) d'un fichier ou d'un répertoire. Les ACL étendent les permissions traditionnelles (propriétaire, groupe, autres) en permettant de spécifier des permissions pour des utilisateurs individuels et des groupes.

```
(stella@K-Rourou)-[~/Documents/Procédures]  
$ sudo apt install acl  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done
```

Seul l'utilisateur
propriétaire à des
droits RW sur le
fichier fic3

```
(stella@K-Rourou)-[~/Documents/Procédures]  
$ sudo getfacl fic3  
# file: fic3  
# owner: stella  
# group: stella  
user::rw-  
group::—  
other::—
```




ACL

Setfacl: permet de modifier les permissions

-m (modifier) : Cette option est utilisée pour ajouter ou modifier une entrée ACL. Elle est suivie de la spécification de l'ACL à ajouter ou modifier, comme `u:utilisateur:rwx` pour un utilisateur ou `g:groupe:r--` pour un groupe.

-x (supprimer) : Permet de supprimer une entrée ACL spécifique. Par exemple, `setfacl -x u:utilisateur fichier` supprime l'ACL de l'utilisateur spécifié sur le fichier.

-d (par défaut) : Utilisée pour définir une ACL par défaut sur un répertoire. Les nouveaux fichiers et sous-répertoires créés dans ce répertoire hériteront de cette ACL. Il faut l'utiliser conjointement avec l'option -m. Par exemple, `setfacl -d -m u:utilisateur:rwx repertoire` définit une ACL par défaut pour l'utilisateur spécifié sur le répertoire.

-R (récursif) : Applique l'ACL à tous les fichiers et sous-répertoires d'un répertoire de manière récursive. Il faut l'utiliser avec -m. Par exemple, `setfacl -R -m u:utilisateur:rwx repertoire` applique l'ACL à tous les fichiers et dossiers à l'intérieur du répertoire spécifié.

-b (supprimer tout) : Supprime toutes les ACL étendues d'un fichier ou répertoire, le ramenant à ses permissions de base.



ACL

Ici je vais ajouter un autre utilisateur que le propriétaire avec les droits RWX sur fic3

```
(stella@K-Rourou)-[~/Documents/Procédures]
$ sudo setfacl -m u:user1:rwx fic3

(stella@K-Rourou)-[~/Documents/Procédures]
$ sudo getfacl fic3
# file: fic3
# owner: stella
# group: stella
user::rw-
user:user1:rwx
group::—
mask::rwx
other::—
```



ACL

Autres exemples

Donner les droits de lecture et d'écriture à un groupe sur un répertoire et tous ses contenus :

```
setfacl -R -m g:mon_groupe:rwX mon_repertoire
```

Supprimer tous les ACL d'un fichier :

```
setfacl -b mon_fichier
```

Appliquer ces ACL à un autre fichier (en supprimant d'abord les ACL existantes) :

```
setfacl -b fichier_destination  
setfacl --set-file=acl_temp.txt fichier_destination
```

```
# Fichier acl_temp.txt  
#file: fichier_source  
# owner: utilisateur  
# group: groupe  
user::rwx  
user:toto:r-x  
group::r-x  
group:compta:rwx  
mask::r-x #droits maximum  
other::r--
```




ACL

```
[stella@Central-Demo ~]$ getfacl fichier_destination
# file: fichier_destination
# owner: stella
# group: stella
user::rw-
group::r--
other::r--

[stella@Central-Demo ~]$ setfa
setfacl  setfattr
[stella@Central-Demo ~]$ setfacl -m g:compta:rwX fichier_destination
[stella@Central-Demo ~]$ getfacl fichier_destination
# file: fichier_destination
# owner: stella
# group: stella
user::rw-
group::r--
group:compta:rwX
mask::rwX
other::r--

[stella@Central-Demo ~]$
```



ACL

```
[stella@Central-Demo ~]$ setfacl -b fichier_destination
[stella@Central-Demo ~]$ getfacl fichier_destination
# file: fichier_destination
# owner: stella
# group: stella
user::rw-
group::r--
other::r--

[stella@Central-Demo ~]$ setfacl --set-file=acl.txt fichier_destination
[stella@Central-Demo ~]$ getfacl fichier_destination
# file: fichier_destination
# owner: stella
# group: stella
user::rwx
user:toto:r-x
group::r-x
group:compta:rwx
mask::r-x
other::r--
#effective:r-x

[stella@Central-Demo ~]$
```

```
[stella@Central-Demo ~]$ cat acl.txt
# Fichier acl_temp.txt
#file: fichier_source
# owner: utilisateur
# group: groupe
user::rwx
user:toto:r-x
group::r-x
group:compta:rwx
mask::r-x #droits maximum
other::r--
[stella@Central-Demo ~]$
```




Mise en situation

10/07/2025



cours.it-system-network.com